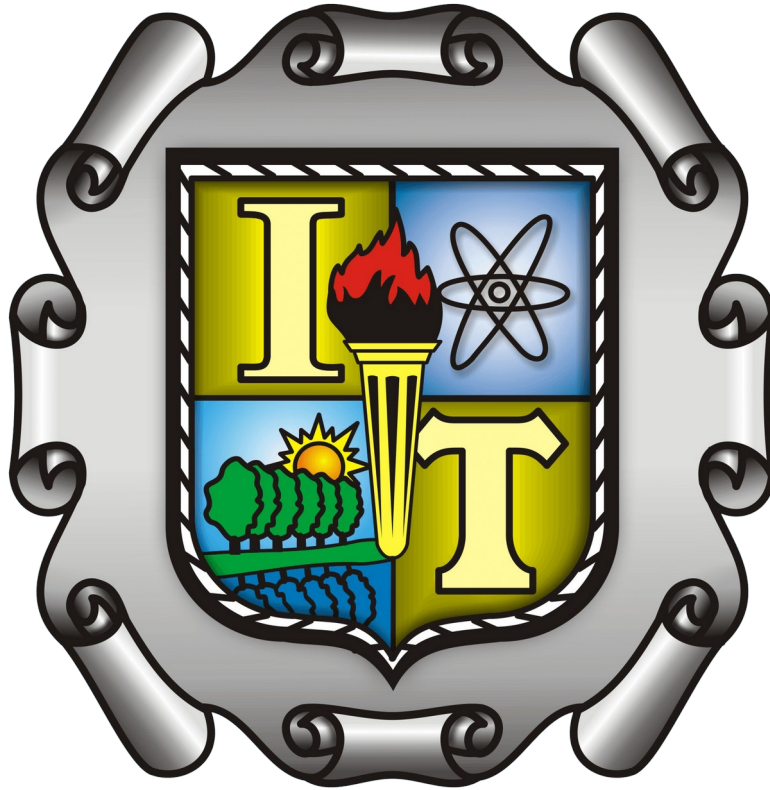


Tecnológico Nacional De México

Instituto Tecnológico de Saltillo



COMPUTO EN LA NUBE

4.2 Demostrar la inyección SQL

NOMBRE: ALEJANDRO FELIPE MACIAS MARTINEZ

NUM.CONTROL: 20051204

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> █
```

se crea una BD donde se guardara los usuarios

```
MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> USE vulnerable_db;
Database changed
MariaDB [vulnerable_db]> CREATE TABLE usuarios (
    -> id INT AUTO_INCREMENT PRIMARY KEY,
    -> nombre_usuario VARCHAR(50),
    -> contrasena VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.014 sec)

MariaDB [vulnerable_db]> █
```

Usa la BD y crea la tabla de usuarios

```
MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('admin', '1234');
Query OK, 1 row affected (0.011 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES ('user', 'password');
Query OK, 1 row affected (0.011 sec)

MariaDB [vulnerable_db]> █
```

inserta datos

```
GNU nano 2.2 login.php
<?php
$conn = new mysqli('localhost', 'test', '1234', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    // CONSULTA VULNERABLE
    $sql = "SELECT * FROM usuarios WHERE nombre_usuario='$usuario' AND contrasena='$contrasena'";

    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}

?>

<form method="POST">
Usuario: <input type="text" name="usuario"><br>
```

crear codigo vulnerable en WHERE nombre_usuario='\$usuario'

Usuario:

Contraseña:

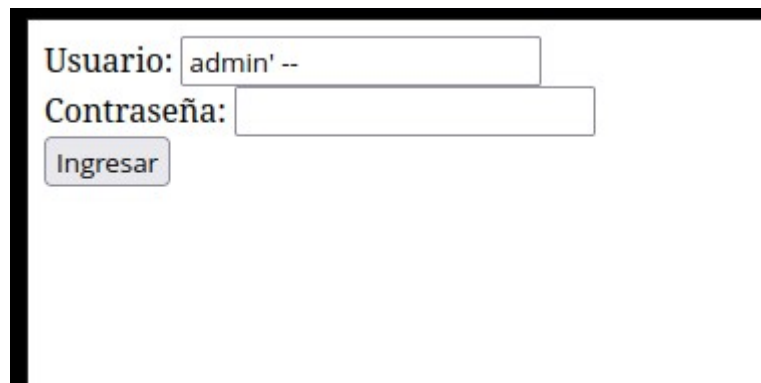
login creado contraseña admin 1234

Login exitoso

Usuario:

Contraseña:

login exitoso

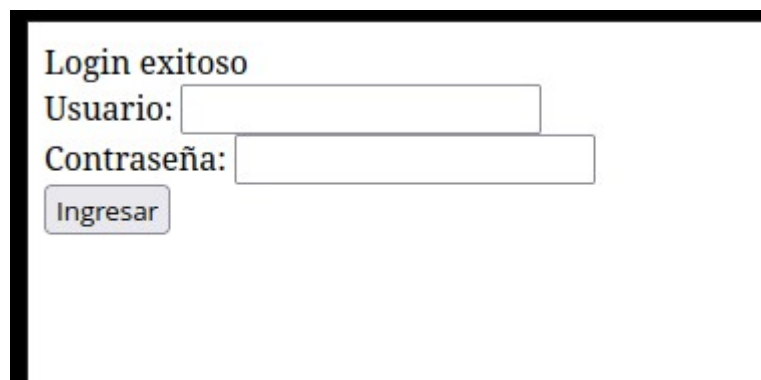


Usuario:

Contraseña:

Login exitoso

inyeccion sql



Usuario:

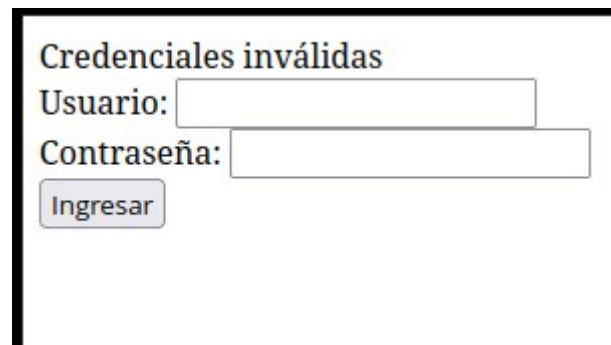
Contraseña:

Login exitoso

tubo un login exitoso Se realizó una inyección SQL utilizando `admin' --`, donde el operador `--` comenta el resto de la consulta SQL, eliminando la validación de la contraseña y permitiendo el acceso sin credenciales válidas.

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
    $usuario = $_POST['usuario'];  
    $contrasena = $_POST['contrasena'];  
  
    // CONSULTA SEGURA  
    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE nombre_usuario=? AND contrasena=?");  
  
    if (!$stmt) {  
        die("Error en prepare: " . $conn->error);  
    }  
  
    $stmt->bind_param("ss", $usuario, $contrasena);  
    $stmt->execute();  
  
    $stmt->store_result();  
  
    if ($stmt->num_rows > 0) {  
        echo "Login exitoso";  
    } else {  
        echo "Credenciales inválidas";  
    }  
}  
?>  
<form method="POST">
```

se modifica el codigo corrigiendo la vulnerabilidad



Credenciales inválidas

Usuario:

Contraseña:

una vez corregido el login la misma inyeccion sql no funciona arrojando credenciales invalidas

En conclusión:

La inyección SQL es una vulnerabilidad crítica que permite manipular consultas a bases de datos. En esta práctica se demostró cómo es posible evadir la autenticación mediante la manipulación de entradas y cómo el uso de consultas preparadas elimina este riesgo.